UNIT 18

API Authentication: OAuth 2.0 and JWT

The OAuth 2.0 Flow | Decoding a JWT | Common JWT Pitfalls

Complete Lecturer Notes and Student Handout

Level: Diploma

> **WARNING:** **Legal and ethical reminder**
>
> Every exploit shown in this unit runs only against the MiniBank lab supplied with these notes, entirely on 127.0.0.1 (your own machine). Forging tokens or bypassing authentication against a real service you do not own, or do not have written permission to test, is a criminal offence under the Computer Misuse and Cybercrimes Act, 2018 (Kenya) and equivalent laws elsewhere.

Lesson Overview

Learning outcomes

By the end of this session, learners should be able to:

1. Explain why APIs need their own authentication and authorisation model, distinct from the cookie-based sessions covered in earlier units.
2. Name the four classic OAuth 2.0 roles and describe the Authorization Code flow step by step.
3. Explain what PKCE is, why it was added, and how it stops an intercepted authorization code from being redeemed by an attacker.
4. Carry out a guided demonstration of a full OAuth 2.0 Authorization Code + PKCE exchange against a working lab authorization server.
5. Describe the structure of a JSON Web Token (header, payload, signature) and decode one by hand, without any library.
6. Explain clearly that a JWT is signed, not encrypted, and state the practical consequence of that distinction.
7. Identify and demonstrate five common JWT verification pitfalls: alg=none, RS256/HS256 algorithm confusion, ignoring expiry, weak HMAC secrets, and missing audience/issuer checks.
8. Implement a JWT verifier that avoids all five pitfalls, and explain why each individual fix matters.
9. Relate OAuth 2.0 and JWT concepts back to the OWASP API Security Top 10 (Unit 6) and to broken access control more generally.

Prerequisites and lab requirements

- **Prior knowledge:** HTTP requests/responses, cookies and sessions, web application architecture (Unit 6), the OWASP Top 10 and its API-specific list (Unit 6), CSRF and the idea of forged requests (Unit 9).
- **Software:** Python 3.9+ with Flask, PyJWT, cryptography, and requests (`pip install flask pyjwt cryptography requests`), plus curl and a text editor.
- **Lab files (supplied with these notes, listed in Appendix A):** `auth_server.py`, `client_app.py`, `resource_server.py`, `run_oauth_demo.py`, `jwt_inspect.py`, `jwt_pitfalls.py`, `make_sample_jwt.py`, `run_lab_demo.sh`, and an `as_private.pem` / `as_public.pem` RSA key pair.
- **Network:** everything runs on **127.0.0.1**, using different ports to stand in for the three separate parties in an OAuth exchange (authorization server, client application, resource server/API).

Quick setup and self-test (run before class)

```
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
pip install flask pyjwt cryptography requests
bash run_lab_demo.sh              # runs the whole unit's demos end to end
```

1. Introduction: Why APIs Need Their Own Authentication

1.1 The problem with cookies, for APIs

Units 6 and 9 covered the classic web pattern: a browser logs in, the server creates a **session** and sets a **cookie**, and the browser sends that cookie back automatically on every later request. This works well for a single website talking to its own server. It breaks down as soon as any of the following are true, which is now the normal case rather than the exception:

- A **mobile app** needs to call a backend API, and has no browser cookie jar.
- A **third-party application** needs limited access to a user's data on another service - for example, a photo-printing site that needs to read (not post to) a user's cloud photo album - **without ever seeing the user's password**.
- A **single-page application** (Unit 6) calls several independent backend services, possibly on different domains, where cookies do not travel cleanly (Unit 9's same-site rules).
- A **microservice** needs to call another microservice on behalf of a user, and needs a portable, verifiable proof of who that user is and what they are allowed to do.

OAuth 2.0 solves the "limited, delegated access without sharing a password" problem. **JWT (JSON Web Token)** is one common, and extremely widely used, format for the tokens that OAuth 2.0 (and many other systems) issue and exchange. They are frequently used together, but they solve two **different** problems - this distinction matters and is revisited throughout this unit.

1.2 Two different questions

Question	Answered by	Covered in
How does an application get permission to act on a user's behalf, without the user handing over their password?	OAuth 2.0 - a delegation / authorization framework	Sections 2–4
How is a claim about identity or permission represented and verified, so any party can check it was really issued by a trusted source and has not been tampered with?	JWT - a token format (not specific to OAuth; also used for session tokens, service-to-service auth, and more)	Sections 5–6

KEY POINT: **A common point of confusion, cleared up early**

OAuth 2.0 is not an authentication protocol - strictly, it is an **authorization** (delegation) framework: it answers "is this application allowed to do X on behalf of this user?" Proving *who the user is* to the authorization server is **OpenID Connect (OIDC)**, a thin identity layer built on top of OAuth 2.0, which this unit mentions but does not build from scratch. What this unit's lab *does* build is the OAuth 2.0 flow that issues an **access token** - and that access token, in our lab (and in a great many real systems), happens to be a JWT.

1.3 Where this unit fits

- **Unit 6 (Web architecture + OWASP)** introduced the application and data tiers, and the API Security Top 10, where **API2 Broken Authentication** and **API5 Broken Function Level Authorization** are exactly the territory of this unit.

- **Unit 9 (CSRF)** showed an attack that exploits a browser automatically attaching a cookie. OAuth/JWT-based APIs are **not automatically vulnerable to CSRF** in the same way, because a bearer token must be deliberately attached by the calling application (not auto-sent by the browser) - but, as this unit shows, they bring an entirely different set of mistakes.
- **Unit 14 (MITM and TLS)** showed that correct certificate validation is what makes a secure channel trustworthy. Everything in this unit assumes that channel exists; OAuth and JWT solve a **different** layer of the problem - proving identity and permission **on top of** a secure channel, not instead of one.

2. OAuth 2.0 Foundations

2.1 The problem OAuth 2.0 was designed to solve

Before OAuth, the only way for a third-party application to access a user's data on another service was the **"password anti-pattern"**: the user typed their actual password for Service A into Service B's login form, and Service B stored it to make API calls as if it were the user. This was dangerous for obvious reasons: Service B could do **anything** the user could do, forever, until the user changed their password everywhere; a breach of Service B leaked the user's real password to Service A as well.

OAuth 2.0 (RFC 6749, published 2012) replaces this with a flow where the user **authenticates directly with the service that holds their data**, and that service then hands the third-party application a **limited, scoped, revocable token** - never the password itself.

2.2 The four roles

Role	Definition	In this unit's lab
Resource Owner	The user who owns the data and can grant access to it	Alice
Client	The application requesting access on the user's behalf (not to be confused with a "client" in the client-server networking sense - here it is the *application*, which can itself be a server)	`client_app.py` (the "MiniBank" third-party app)
Authorization Server	Issues tokens after authenticating the resource owner and obtaining their consent	`auth_server.py`
Resource Server	Hosts the protected data/API and accepts tokens as proof of authorised access	`resource_server.py`

In many real systems the Authorization Server and Resource Server belong to the same organisation (for example, a bank's own login service and its own account API) - exactly as modelled in this unit's lab - but they are logically separate **roles**, and large identity providers (Google, Microsoft Entra ID, Auth0, Okta) often act as the Authorization Server for many *different* Resource Servers belonging to many different companies.

2.3 Key terms

Term	Meaning
Scope	A named, limited permission the client is requesting (for example `accounts:read`); the user consents to specific scopes, not blanket access
Authorization code	A short-lived, single-use code the authorization server gives the client after the user approves, to be exchanged for a token
Access token	The credential the client presents to the resource server on every API call; typically short-lived (minutes, not days)
Refresh token	A longer-lived credential the client can use to obtain a new access token without the user logging in again; must be stored and protected with extra care

Term	Meaning
Redirect URI	The exact, pre-registered URL the authorization server sends the user back to with the code; must match exactly to prevent the code being redirected elsewhere
Bearer token	A token that grants access to **whoever presents it** (like a cinema ticket, not like a photo ID) - this is why protecting the token itself, in transit and at rest, matters so much

2.4 OAuth 2.0 grant types, briefly

OAuth 2.0 defines several "grant types" (ways of obtaining a token) for different situations. This unit focuses on the one that matters most today:

Grant type	Used for	Status today
Authorization Code (+ PKCE)	Any client that can redirect a user through a browser: web apps, mobile apps, single-page apps	**The recommended, default flow for almost everything** - this unit's focus
Client Credentials	Machine-to-machine access with no end user involved at all (a backend job calling another backend service)	Still standard for service-to-service calls
Refresh Token	Exchanging a refresh token for a new access token once the first one expires	Standard, used alongside the flows above
Implicit	Was once recommended for browser-based apps that could not keep a secret (returned the token directly in the URL fragment, no code exchange step)	**Deprecated** - tokens exposed in URLs/browser history were a security weakness; Authorization Code + PKCE replaces it even for public clients
Resource Owner Password Credentials (ROPC)	The client collects the user's password directly and exchanges it for a token	**Deprecated / discouraged** - this is exactly the "password anti-pattern" OAuth was meant to avoid, just moved one layer down

3. The Authorization Code Flow with PKCE, Step by Step

This is the flow built in this unit's lab, and the one almost every modern application should use. We walk through it twice: once as a diagram, once against the **actual, running code** in Appendix A.

3.1 Why PKCE?

PKCE (Proof Key for Code Exchange, pronounced "pixy", RFC 7636) closes a specific gap in the plain Authorization Code flow: the authorization **code** travels through the user's browser (as part of a redirect URL) on its way back to the client. On a shared or compromised device, or through a badly behaved app that can intercept redirects, that code could potentially be captured by something other than the real client. Without PKCE, whoever has the code can exchange it for a token. With PKCE, the client additionally proves it is the **same party that started the flow**, using a secret only it knows.

1. **Before** sending the user anywhere, the client generates a random secret (the **code verifier**) and computes its SHA-256 hash (the **code challenge**).
2. The client sends only the **challenge** (never the verifier) in the initial authorization request.
3. When exchanging the returned code for a token, the client must also send the original **verifier**.
4. The authorization server hashes the verifier it just received and checks it matches the challenge it was given at the start. **Only the real client, which generated the verifier in step 1 and never sent it anywhere until step 3, can pass this check.**

An attacker who steals the authorization code (for example, from a poorly secured redirect, a shared device's browser history, or a malicious app intercepting the callback) **cannot redeem it**, because they do not have the verifier - it never left the legitimate client until the very last, direct, server-to-server step.

3.2 The flow, as a diagram

Figure 3.1: Authorization Code flow with PKCE



```
          |                                    |                        |
|
          |                                    |---- 5. GET /accounts  Authorization: Bearer
<token> ---->|
          |                                    |<---------------------- protected data -------
-------------|
```

3.3 Step-by-step against the real lab code

Step 1 - the client prepares PKCE and redirects the user

```
# client_app.py
verifier = base64.urlsafe_b64encode(secrets.token_bytes(32)).rstrip(b"=").decode()
challenge =
base64.urlsafe_b64encode(hashlib.sha256(verifier.encode()).digest()).rstrip(b"=").decode()
session["pkce_verifier"] = verifier     # kept server-side, NEVER sent to the browser as a URL
parameter

params = {
    "client_id": CLIENT_ID, "redirect_uri": REDIRECT_URI, "state": state,
    "code_challenge": challenge, "code_challenge_method": "S256",
    "scope": "accounts:read", "response_type": "code",
}
return redirect(f"{AUTH_SERVER}/authorize?{urllib.parse.urlencode(params)}")
```

Notice the **`state`** parameter too: a random value the client generates and later checks matches on the way back - this is the OAuth flow's own CSRF protection (Unit 9), stopping an attacker from tricking a victim into completing *the attacker's* OAuth flow.

Step 2 - the authorization server authenticates the user and gets consent

```
# auth_server.py
@app.route("/authorize", methods=["GET", "POST"])
def authorize():
    if request.method == "GET":
        ...
        if code_challenge_method != "S256" or not code_challenge:
            return "PKCE required: code_challenge (S256) missing", 400
        return render_template_string(LOGIN_PAGE, ...)   # shows login + consent screen

    # POST: user has logged in and approved
    code = secrets.token_urlsafe(24)
    AUTH_CODES[code] = {
        "client_id": ..., "redirect_uri": ..., "code_challenge": ...,
        "user": u, "scope": ..., "expires": time.time() + 60, "used": False,
    }
    return redirect(f"{request.form['redirect_uri']}?code={code}&state=...")
```

The authorization server **refuses to even show the login page** without a valid PKCE challenge - PKCE is enforced at the very first step, not bolted on later. The code it eventually issues is **short-lived (60 seconds)** and marked **single-use**.

Step 3–4 - the client exchanges the code for a token

```
# auth_server.py  (the /token endpoint)
entry = AUTH_CODES.get(code)
if not entry or entry["used"] or entry["expires"] < time.time():
    return jsonify(error="invalid_grant", ...), 400
if entry["client_id"] != client_id or entry["redirect_uri"] != redirect_uri:
```

```
    return jsonify(error="invalid_grant", ...), 400

# --- PKCE verification ---
computed =
base64.urlsafe_b64encode(hashlib.sha256(verifier.encode()).digest()).rstrip(b"=").decode()
if computed != entry["code_challenge"]:
    return jsonify(error="invalid_grant", error_description="PKCE verification failed"), 400

entry["used"] = True   # one-time use, even if the verifier WAS correct

access_token = jwt.encode(access_payload, PRIVATE_KEY, algorithm="RS256", headers={"kid": KID})
return jsonify(access_token=access_token, token_type="Bearer", expires_in=300,
scope=entry["scope"])
```

Three independent checks all have to pass: the code must be unused and unexpired, the `client_id`/`redirect_uri` must match what was originally registered for this code, **and** the PKCE verifier must hash to the original challenge. This "all of the above" design is deliberate defence in depth (Unit 9's terminology applies directly here too).

Step 5 - the client calls the protected API

```
# client_app.py
resp = requests.get("http://127.0.0.1:7002/accounts",
                    headers={"Authorization": f"Bearer {token}"})
```

The **access token itself** (a JWT, in this lab) is now the client's proof of identity and permission for every call to the resource server. Section 5 pulls that token apart to see what it actually contains, and Section 6 shows what goes wrong when the resource server checks it carelessly.

4. Demo 1: The OAuth 2.0 + PKCE Flow, Live

DEMO: DEMO 1: the OAuth 2.0 + PKCE flow, live

Verified output, captured while preparing this unit

```
$ python auth_server.py &
$ python resource_server.py &
$ python run_oauth_demo.py

[1] Client generates PKCE verifier/challenge and sends the user to /authorize
    GET /authorize -> 200 (login/consent page shown)

[2] User logs in and approves -- authorization server issues a one-time code
    POST /authorize -> 302, redirected with code=uXmf_TnPmMmh...

[3] Client exchanges the code for an access token, proving it holds the PKCE verifier
    POST /token -> 200
    access_token = eyJhbGciOiJSUzI1NiIsImtpZCI6ImFzLWtleS0xIiwidHlwIj...

[4] Client calls the protected API with the access token
    GET /accounts -> 200  {'accounts': [{'balance': 1000, 'id': 'AC-001'}], 'scope':
'accounts:read', 'user': 'alice'}

[5] Attacker intercepts a FRESH authorization CODE and tries to redeem it without the (secret,
browser-held) verifier
    POST /token (correct fresh code, WRONG verifier) -> 400  {'error': 'invalid_grant',
'error_description': 'PKCE verification failed'}

[6] Legitimate client tries to reuse its OWN already-spent code a second time
    POST /token (code reuse) -> 400  {'error': 'invalid_grant', 'error_description': 'code
unknown, used, or expired'}

Demo complete: PKCE blocked a stolen/guessed code, and the code was single-use.
```

4.1 Protecting tokens once issued

- **Always use HTTPS** for every leg of the flow (Unit 14) - PKCE protects the *code*, but the access token itself, and the code before exchange, must still travel over an encrypted, validated channel.
- **Store tokens carefully.** In a browser-based app, prefer an `HttpOnly` cookie (Unit 9) over `localStorage`, specifically because JavaScript - and therefore an XSS payload (Unit 6) - cannot read an `HttpOnly` cookie, but can trivially read `localStorage`.
- **Keep access tokens short-lived** (our lab: 5 minutes) and use refresh tokens, stored even more carefully, to get new ones - this limits the damage window if a token does leak.
- **Validate the exact `redirect_uri`** server-side against a pre-registered list (our lab does this in `/authorize`) - never accept a wildcard or pattern match, which is a known class of OAuth misconfiguration.

5. JSON Web Tokens (JWT)

5.1 What is a JWT?

A **JSON Web Token (JWT, RFC 7519)** is a compact, URL-safe way of representing a set of claims (statements about a subject - typically a user) that can be **verified and trusted**, because it is **digitally signed**. A JWT is just text: three base64url-encoded sections separated by dots.

Figure 5.1: a real JWT issued by this unit's lab (line-broken for readability; it is actually one line)

```
eyJhbGciOiJSUzI1NiIsImtpZCI6ImFzLWtleS0xIiwidHlwIjoiSldUIn0.
eyJpc3MiOiJodHRwczovL2F1dGgubWluaWJhbmsubG9jYWwiLCJzdWIiOiJ1c2VyLTEwMDEi...
JdnYBFBebtSW18uqvlY8e2vQP8c9LXmqp3fleLjCL3qseCtwFiEDXn0zuokEdRxKNDSfPwzn...

\_____________ HEADER _____________/\_____________ PAYLOAD _____________/\___ SIGNATURE
___/
```

5.2 The three parts

Part	Content	Purpose
Header	Algorithm (`alg`) and key identifier (`kid`), plus token type (`typ`)	Tells the verifier how the token was signed and, usually, which key to check it against
Payload	The **claims**: `iss` (issuer), `sub` (subject/user), `aud` (audience), `iat`/`exp` (issued-at / expiry), `scope`, and any application-specific data	The actual statements being made, in plain JSON
Signature	A cryptographic signature over the header and payload, using the algorithm in the header	Proves the header and payload have not been altered since signing, and (for asymmetric algorithms) who signed it

5.3 Signed, not encrypted - the single most important fact about JWT

> **WARNING: The one-sentence version**
> Base64url is an **encoding**, not **encryption**. **Anyone who has the token can read the header and payload**, instantly, with no key at all. The signature proves the token is **genuine and unmodified** - it does **not** hide the contents. **Never put a secret (a password, a credit card number, a private detail the user hasn't consented to expose) inside a JWT payload.**

This single fact explains both halves of JWT security: the payload is **public** (so never put secrets in it), and the signature is the **only** thing standing between a legitimate token and a forged one (so verifying it correctly, which Section 6 is entirely about, is everything).

> **KEY POINT: If true confidentiality of the payload is needed**
> A related standard, **JWE (JSON Web Encryption, RFC 7516)**, encrypts the payload instead of just signing it. It is far less commonly used than signed JWTs (JWS) for ordinary API access tokens, precisely because access tokens are usually meant to be *readable* by the resource server (and often by the client) - just not *forgeable*.

5.4 Demo: decoding a JWT by hand

DEMO: DEMO 2: decode a JWT with no library at all

This demo has one purpose: to make Section 5.3's warning viscerally obvious by actually reading a token's contents using nothing but base64 decoding.

Verified output, captured while preparing this unit

```
$ python jwt_inspect.py

=== HEADER (base64url-decoded, anyone can read this) ===
{
  "alg": "RS256",
  "kid": "as-key-1",
  "typ": "JWT"
}

=== PAYLOAD (base64url-decoded, anyone can read this) ===
{
  "iss": "https://auth.minibank.local",
  "sub": "user-1001",
  "aud": "https://api.minibank.local",
  "iat": 1793013765,
  "exp": 1793014065,
  "scope": "accounts:read",
  "name": "Alice"
}

=== SIGNATURE ===
256 raw bytes -- this is the ONLY part that proves the token is genuine

NOTE: base64url is an ENCODING, not ENCRYPTION. No key was needed
to read everything above. A JWT's contents are PUBLIC; only its
SIGNATURE is secret. Never put secrets you don't want read inside a JWT.
```

The script doing this is deliberately tiny:

The entire "attack" - four lines of standard-library Python, no JWT library, no key

```
def b64url_decode(segment: str) -> bytes:
    padding = "=" * (-len(segment) % 4)
    return base64.urlsafe_b64decode(segment + padding)

header_b64, payload_b64, signature_b64 = token.split(".")
header = json.loads(b64url_decode(header_b64))
payload = json.loads(b64url_decode(payload_b64))
```

CLASS ACTIVITY: In-class activity (5 minutes)

Copy the access token printed by `run_oauth_demo.py` (Demo 1) and run `python jwt_inspect.py <that token>` on it. Identify the `scope`, `exp` and `sub` claims, and calculate (by hand, from the Unix timestamps) how many seconds the token is valid for. Then answer: "If you were an attacker who intercepted this token in transit, what have you just learned, even without being able to use it?" Expected answer: who the user is, what they can do, which system issued it, and exactly when it expires - useful reconnaissance even before considering whether the token itself can be reused or forged.

A browser extension or a site such as **jwt.io** does exactly this decoding for convenience during development - and, just like our script, it needs no key and proves nothing about whether the signature is actually valid. **Decoding is not the same as verifying.** Section 6 is about verifying.

5.5 Symmetric vs asymmetric signing

	HS256 (HMAC + SHA-256)	RS256 (RSA + SHA-256)
Key type	One shared secret	A private key (signs) and a public key (verifies) - a key *pair*
Who can sign	Anyone with the shared secret	Only the holder of the private key
Who can verify	Anyone with the shared secret (same party that can sign!)	Anyone with the public key - which can be freely published
Typical use	A single application signing and verifying its own tokens	An authorization server signs; many independent resource servers verify, without ever holding the signing key
Used in this unit's lab	Only inside Section 6's pitfall demonstrations, to show what goes wrong	`auth_server.py` and `resource_server.py` - the real OAuth flow in Demo 1

This distinction - "with HS256, the party that *verifies* a token could also have *signed* one" - is exactly what Section 6, Pitfall 2 exploits.

5.6 JWKS: publishing the public key

For RS256 (and other asymmetric algorithms), the authorization server publishes its public key(s) at a well-known URL, typically `/.well-known/jwks.json`, in **JSON Web Key Set (JWKS)** format. A resource server fetches this once (and caches it), then uses the `kid` (key ID) in each token's header to pick the right key if multiple are published - which is exactly how key rotation works without downtime: publish the new key alongside the old one, start signing with the new `kid`, and only remove the old key once every outstanding token signed with it has expired.

```
# auth_server.py
@app.route("/.well-known/jwks.json")
def jwks():
    # Expose the PUBLIC key only -- resource servers verify signatures
    # without ever holding the authorization server's private key.
    ...
    jwk = {"kty": "RSA", "kid": KID, "use": "sig", "alg": "RS256", "n": ..., "e": ...}
    return jsonify(keys=[jwk])

# resource_server.py
def get_signing_key(kid):
    jwks_doc = requests.get(f"{AUTH_SERVER}/.well-known/jwks.json").json()
    for k in jwks_doc["keys"]:
        if k["kid"] == kid:
            return jwt.PyJWK(k).key
```

6. Common JWT Pitfalls

Every pitfall below is demonstrated in `jwt_pitfalls.py` as a **genuinely working exploit** against a deliberately vulnerable verifier, immediately followed by the same forged token failing against a fixed, secure verifier. This is the heart of the unit: JWT *signing* is cryptographically strong; almost every real-world JWT vulnerability is a mistake in how the token is **verified**, not a flaw in the cryptography itself.

6.1 Pitfall 1: alg=none

The JWT specification technically allows an **`alg: "none"`** header, meaning "this token is unsecured - do not check any signature." It exists for narrow, specific cases (for instance, a token already protected by another layer). A verifier that blindly trusts the `alg` field **in the token itself** to decide how to check it is catastrophic: an attacker simply writes `"alg": "none"`, leaves the signature segment empty, and fills the payload with whatever claims they like.

```
def vulnerable_verify(token):
    # BUG: decodes using whatever algorithm the TOKEN ITSELF claims in its header
    header = jwt.get_unverified_header(token)
    return jwt.decode(token, options={"verify_signature": False}) if header["alg"] == "none" \
        else jwt.decode(token, PUBLIC_KEY, algorithms=[header["alg"]])

# Attacker forges a token with alg=none and an EMPTY signature, granting themselves admin
forged_token = f"{header_b64}.{payload_b64}."     # note: empty signature segment
```

Verified output

```
  VULNERABLE verifier: ACCEPTED forged token -> role=admin  *** EXPLOIT WORKS ***
  SECURE verifier: REJECTED forged token (InvalidAlgorithmError: The specified alg value is not
allowed)  -- attack blocked
```

The fix is simple and absolute: the **server**, never the token, decides which algorithm(s) are acceptable.

```
def secure_verify(token):
    # FIX: the ALLOWED algorithm list is fixed by the SERVER, never read from the token
    return jwt.decode(token, PUBLIC_KEY, algorithms=["RS256"])
```

6.2 Pitfall 2: RS256/HS256 algorithm confusion ("key confusion")

This is the cleverest and most instructive pitfall. Recall from Section 5.6 that an RS256 **public** key is, by design, meant to be published and freely shared. Now suppose a verifier will accept **either** RS256 or HS256, and - critically - uses the **same key material** (the PEM-formatted public key string) as the verification key for both. For HS256, that string is treated as an **HMAC secret**. Because the "secret" is actually public, **anyone** can use it to sign a token with HS256, and a confused verifier will accept it.

```
def vulnerable_verify_by_hand(token, public_key_pem):
    # BUG: trusts the token's OWN header to decide how to verify it, and for "HS256"
    # treats the (public, publishable) RSA key bytes as if they were a private HMAC secret
    header = json.loads(base64.urlsafe_b64decode(h_b64 + "=="))
    if header["alg"] == "HS256":
        expected = hmac.new(public_key_pem.encode(), signing_input, hashlib.sha256).digest()
        if not hmac.compare_digest(sig, expected):
            raise jwt.InvalidSignatureError("bad HMAC signature")
    ...
```

```
# Attacker signs a forged token with HS256, using the PUBLISHED public key as the HMAC "secret"
forged_sig = hmac.new(public_key_bytes, signing_input, hashlib.sha256).digest()
```

Verified output

```
  VULNERABLE hand-rolled verifier: ACCEPTED forged token -> role=admin  *** EXPLOIT WORKS ***
  SECURE verifier: REJECTED forged token (InvalidAlgorithmError: The specified alg value is not
allowed)  -- attack blocked
```

The fix is the same principle as Pitfall 1: pin the algorithm **list** to exactly the one(s) the server actually uses, and never mix symmetric and asymmetric algorithms for the same key material.

6.3 Pitfall 3: ignoring token expiry

JWT libraries check the `exp` claim **by default** - but that check can be, and in real codebases sometimes is, explicitly switched off, usually "temporarily" during development or debugging, and then never switched back on.

```
def vulnerable_verify(token):
    # BUG: verify_exp explicitly turned off (seen in real code "to make testing easier")
    return jwt.decode(token, PUBLIC_KEY, algorithms=["RS256"], options={"verify_exp": False})
```

Verified output

```
  VULNERABLE verifier: ACCEPTED an hour-old EXPIRED token -> sub=alice  *** EXPLOIT WORKS ***
  SECURE verifier: REJECTED expired token (Signature has expired)  -- attack blocked
```

The fix is simply: do not disable `verify_exp` (and keep access tokens genuinely short-lived, as in Section 4.1, so that even a brief lapse has a small blast radius).

6.4 Pitfall 4: weak, guessable HMAC secrets

When HS256 **is** the right choice (a single application signing and verifying its own tokens - see Section 5.5), the security of every token depends entirely on the secrecy and strength of **one shared value**. A short, memorable, or previously-leaked secret can simply be **guessed**.

```
WEAK_SECRET = "secret123"
token = jwt.encode({"sub": "alice", "role": "user", "exp": ...}, WEAK_SECRET,
algorithm="HS256")

candidates = ["password", "123456", "changeme", "minibank", "letmein", "secret123", "admin"]
for guess in candidates:
    try:
        jwt.decode(token, guess, algorithms=["HS256"])
        cracked = guess; break
    except jwt.InvalidSignatureError:
        continue
```

Verified output

```
  A token signed with a short, common HS256 secret: eyJhbGciOiJIUzI1NiIs...
  Secret CRACKED from a 7-word list: 'secret123'
  Attacker now MINTS their own valid token -> role=admin  *** EXPLOIT WORKS ***
```

A real attacker would use a large wordlist and a fast cracking tool (such as `hashcat` with its dedicated JWT mode) rather than seven hand-typed guesses, but the principle - and the outcome - is identical. **The fix:** use

a long, high-entropy secret generated by a cryptographically secure random generator (32+ random bytes, never a memorable phrase), or, better still, prefer asymmetric signing (RS256/ES256) so the party that *verifies* tokens never needs to hold anything secret at all.

6.5 Pitfall 5: missing audience/issuer checks ("token confusion")

A signature proves a token is genuine and untampered. It does **not**, by itself, prove the token was **meant for the service checking it**. If an organisation has several APIs sharing one authorization server, a perfectly genuine token issued for, say, a *notifications* service could be replayed against an *accounts* service, if the accounts service never checks **who the token was actually issued for**.

```
def vulnerable_verify(token):
    # BUG: a developer hit PyJWT's "Invalid audience" error during testing (because no
    # audience= was supplied) and "fixed" it by turning the check off -- a very real,
    # very common pattern -- instead of supplying the correct expected audience.
    return jwt.decode(token, PUBLIC_KEY, algorithms=["RS256"], options={"verify_aud": False})
```

Verified output

```
  VULNERABLE verifier (accounts API): ACCEPTED a token meant for
'https://notifications.minibank.local'  *** EXPLOIT WORKS (token confusion) ***
  SECURE verifier: REJECTED mismatched-audience token (Audience doesn't match)  -- attack
blocked
```

> **WARNING: This specific mistake is extremely realistic**
>
> PyJWT raises an error if a token has an `aud` claim and the caller did not supply the `audience=` parameter - a genuinely helpful, secure-by-default design. The realistic bug is not "nobody thought about audiences"; it is a developer hitting that error during testing and silencing it with `options={"verify_aud": False}` to "get past" an error they did not fully understand, exactly the same pattern as disabling certificate validation in Unit 14. **The lesson generalises: an error from a security library is usually the library doing its job, not a bug to be silenced.**

The fix: always pass the expected `audience` (and `issuer`) explicitly, and never resolve a validation error by disabling the check that raised it.

```
def secure_verify(token):
    return jwt.decode(token, PUBLIC_KEY, algorithms=["RS256"],
                      audience="https://api.minibank.local",
                      issuer="https://auth.minibank.local")
```

6.6 A secure JWT verification checklist

This is exactly what `resource_server.py` (Appendix A) implements, and exactly what defeats every pitfall above at once:

Every numbered comment corresponds to one pitfall above

```
claims = jwt.decode(
    token, key,
    algorithms=["RS256"],          # 1. PIN the algorithm -- never trust the token's own header
(Pitfalls 1, 2)
    audience=EXPECTED_AUDIENCE,    # 2. this token must be FOR this API (Pitfall 5)
    issuer=EXPECTED_ISSUER,        # 3. and must come FROM our trusted authorization server
(Pitfall 5)
)
```

```
# 4. exp is checked by DEFAULT -- never pass verify_exp: False (Pitfall 3)
# 5. the signing key is RS256 (asymmetric), fetched by kid from JWKS -- never a short/weak
HS256 secret (Pitfall 4)
```

DEMO: DEMO 3: exploit, then fix, all five pitfalls

Run `python jwt_pitfalls.py`: "What exactly did the attacker exploit, and which line of the secure verifier fixes it?" Open `jwt_pitfalls.py`, comment out one line of a `secure_verify` function at a time (for example, removing `audience=...`), and confirm the corresponding attack starts working again - turning the checklist in Section 6.6 from something to memorise into something you have personally broken and fixed.

7. Real-World Relevance

> REAL-LIFE SCENARIO: **JWT "alg=none" and library defaults (industry-wide pattern, 2015 onward)**
>
> When JWT libraries were new, several popular implementations across different languages defaulted to **trusting the `alg` field in the token** to decide how to verify it - exactly Pitfall 1. Security researchers demonstrated that an attacker could take any genuine RS256 token, strip it down to `alg: "none"` with an empty signature, and have it accepted by a careless verifier. The fix (pinning the server-side allowed algorithm list, exactly as `secure_verify` does in Section 6.1) is now standard advice in every major library's documentation, and most libraries now require the allowed algorithms to be stated explicitly - but applications written against older guidance, or that hand-roll verification, remain at risk, which is precisely why this is taught as a *verification* mistake rather than a *library* flaw.

> REAL-LIFE SCENARIO: **OAuth redirect_uri and token-leakage vulnerabilities (recurring pattern)**
>
> Security researchers and bug-bounty reports repeatedly find OAuth implementations that validate the `redirect_uri` loosely - accepting any URL on the same domain, or matching only a prefix, rather than an exact string. An attacker who controls *any* page on that domain (an open redirect, a forgotten subdomain, a permissive path) can redirect the authorization code or token to themselves. This is why this unit's lab (Section 3.3, Step 2) checks `redirect_uri` for an **exact match** against a pre-registered list, and why PKCE (Section 3.1) exists as a second, independent layer even when a code is briefly exposed.

> REAL-LIFE SCENARIO: **Weak JWT secrets found in real applications (recurring pattern)**
>
> Penetration testers and bug-bounty hunters routinely find production JWTs signed with short, dictionary-guessable HS256 secrets - sometimes the literal word "secret", a company name, or a default value copied from a tutorial and never changed. Publicly available wordlists exist specifically for this purpose, and tools such as `hashcat` include a dedicated JWT-cracking mode. Section 6.4's seven-word demonstration is a miniature, classroom-safe version of a genuinely common finding.

None of these require breaking cryptography. Every one of them is a **verification logic** mistake - which is exactly why Section 6's checklist, not a deeper mathematics lesson, is the practical takeaway of this unit.

8. Activities, Discussion and Assessment

8.1 Hands-on lab: Run, Break, and Rebuild

Using the supplied files (Appendix A), entirely on `127.0.0.1`.

1. Run Demo 1 (`run_oauth_demo.py`) manually and capture the full output, including both failed token-exchange attempts.
2. Run `jwt_inspect.py` against the access token your own run of Demo 1 produced. Record the `scope`, `aud` and `exp` claims, and calculate how many seconds remained before expiry at the moment you captured it.
3. Run `jwt_pitfalls.py` and, for **each** of the five pitfalls, write one sentence stating exactly what the vulnerable verifier failed to check.
4. Open `resource_server.py` and temporarily make one change that reintroduces **one** pitfall of your choice (for example, remove the `audience=` argument). Restart the resource server, rerun the relevant step of `run_oauth_demo.py` or a crafted request, and confirm - and screenshot - that the access control gap reopens. **Then put the fix back and confirm it closes again.**
5. Using `jwt_inspect.py`, decode (without any key) a JWT you find in a real application you use day to day (your student portal, a web app, anything with a visible `Authorization: Bearer ...` header in browser DevTools - do **not** attempt to use or modify it, only read it). Identify which claims it contains and reflect on whether anything sensitive appears in the payload.
6. Write a short report: the five pitfalls, your one-sentence summary of each, the result of your "break it, then fix it" experiment, and what you found (if anything) in the real-world JWT you inspected.

8.2 Discussion questions

1. Explain, in your own words, why OAuth 2.0 is described as an authorization framework rather than an authentication protocol, and where OpenID Connect fits in.
2. A colleague says, "We don't need PKCE, our client has a confidential client secret." Using Section 3.1 and the OAuth 2.1 note in Section 2.4, evaluate this claim.
3. Why is `state` in the OAuth flow conceptually similar to the anti-CSRF token from Unit 9? What specific attack does it prevent?
4. Explain why "a JWT is signed, not encrypted" has a direct, practical consequence for what a developer should and should not put in a token's payload.
5. Why does accepting *either* RS256 or HS256 in the same verifier create a vulnerability, even if the server only ever *intends* to issue RS256 tokens?
6. A teammate "fixes" a confusing PyJWT audience error by adding `options={"verify_aud": False}`. Explain what has actually happened, using the same reasoning Unit 14 applied to disabled certificate validation.
7. Design a short code-review checklist (five items) for spotting unsafe JWT verification before it reaches production.

8.3 Quick quiz (multiple choice)

Use as a 10-minute classroom quiz. The answer key is in Section 8.5.

Q1. OAuth 2.0 is best described as:

A. An encryption algorithm
B. A delegated authorization framework, not itself an authentication protocol
C. A replacement for HTTPS
D. A type of database

Q2. In the four OAuth roles, who is the "Client"?

A. The user
B. The application requesting access on the user's behalf
C. The authorization server
D. The resource server

Q3. What does PKCE specifically prevent?

A. Weak passwords
B. An intercepted authorization code being redeemed by someone other than the client that started the flow
C. SQL injection
D. Expired certificates

Q4. In PKCE, what does the client send in the FIRST request (to /authorize)?

A. The code_verifier
B. The code_challenge (never the verifier)
C. The access token
D. The client's private key

Q5. Why is the OAuth "state" parameter important?

A. It stores the user's password
B. It protects the OAuth flow itself against CSRF-style attacks
C. It encrypts the access token
D. It sets the token's expiry time

Q6. A JWT's header and payload are:

A. Encrypted and unreadable without the key
B. Base64url-ENCODED and readable by anyone who has the token
C. Stored only on the server
D. Hashed and irreversible

Q7. What does a JWT's signature actually prove?

A. That the payload is secret
B. That the token is genuine and has not been altered since signing
C. That the user has a strong password
D. That the connection is using HTTPS

Q8. The "alg=none" pitfall succeeds because:

A. The verifier lets the TOKEN'S OWN header decide how it should be checked
B. RSA keys are too short
C. The password was weak
D. The token had expired

Q9. Why is accepting both RS256 and HS256 in one verifier dangerous?

A. It is slower
B. The RS256 public key, meant to be shared, can be reused as an HS256 "secret" to forge tokens
C. HS256 is always broken
D. It only works over HTTP

Q10. What is the correct response to a JWT library raising an "Invalid audience" error?

A. Disable the audience check
B. Supply the correct expected audience value
C. Switch to alg=none
D. Ignore the error and retry

Q11. A weak HMAC secret like "secret123" is dangerous mainly because:

A. It is too long
B. It can be guessed or brute-forced, letting an attacker forge valid tokens
C. HMAC is not a real algorithm
D. It only affects RS256 tokens

Q12. Which combination correctly defeats ALL FIVE pitfalls in Section 6 at once?

A. Disabling signature verification for speed
B. Pinning the algorithm list, checking exp (default), and validating audience + issuer, with a strong/asymmetric key
C. Using the shortest possible secret
D. Accepting any algorithm the token specifies

Q13. Why does this unit recommend storing browser-held tokens in an HttpOnly cookie rather than localStorage?

A. HttpOnly cookies are faster
B. JavaScript, and therefore an XSS payload, cannot read an HttpOnly cookie
C. localStorage cannot store strings
D. Cookies never expire

Q14. What is the key practical difference between a JWT (JWS, signed) and JWE (encrypted)?

A. There is no difference
B. A signed JWT's payload is readable by anyone who has it; JWE additionally hides the payload's contents
C. JWE is faster to verify
D. JWT cannot carry an expiry claim

Q15. Decoding a JWT with a tool like jwt.io, without checking the signature, tells you:

A. That the token is definitely valid and safe to trust
B. What the token claims to contain -- nothing about whether that claim is genuine
C. The server's private key
D. Nothing at all

8.4 Short-answer and applied questions

1. Define OAuth 2.0 and explain, with an example, the "password anti-pattern" it was designed to replace. (4 marks)
2. List the four OAuth 2.0 roles and give one real example of each (not from this unit's lab). (4 marks)
3. Describe, step by step, how PKCE prevents a stolen authorization code from being redeemed by an attacker. (6 marks)
4. Explain the structure of a JWT (three parts) and state, with justification, whether each part is readable without a key. (5 marks)
5. Pick any TWO of the five JWT pitfalls from Section 6. For each, explain the vulnerable behaviour, show (in words) how an attacker exploits it, and state the fix. (10 marks)
6. A resource server accepts tokens signed with either RS256 or HS256, using the same public key string for both. Explain exactly why this is dangerous, and propose a one-line fix. (5 marks)

9. Summary, Glossary and Quick Reference

9.1 Key takeaways

- **OAuth 2.0 and JWT solve two different problems:** OAuth 2.0 is about **delegated authorization** ("can this app act on my behalf, for these specific things, without my password?"); JWT is a **token format** for representing verifiable claims, which OAuth commonly uses to carry its access tokens.
- The **Authorization Code flow with PKCE** is the recommended pattern for essentially every client type today - web apps, mobile apps, and single-page applications alike - and this unit's lab builds and runs it in full, including the authorization code's single-use and short lifetime as independent, defence-in-depth protections alongside PKCE itself.
- PKCE works because the client proves it holds a secret (**the verifier**) that only it generated, by showing the server can derive that secret's hash (**the challenge**) back from it - an attacker who only intercepts the authorization code never has the verifier.
- A JWT is **signed, not encrypted**: its header and payload are plain, readable, base64url-encoded JSON. **Never put secrets in a JWT payload.**
- Essentially all real-world JWT vulnerabilities are **verification mistakes**, not flaws in the underlying cryptography: trusting the token's own `alg` header (Pitfall 1), mixing symmetric and asymmetric algorithms on the same key material (Pitfall 2), disabling expiry checks (Pitfall 3), using weak shared secrets (Pitfall 4), and skipping or disabling audience/issuer checks (Pitfall 5).
- A secure verifier **pins the algorithm**, **never disables the default expiry check**, **validates audience and issuer explicitly**, and, wherever possible, **prefers asymmetric signing (RS256/ES256)** so the verifying party never needs to hold a secret at all.
- A security library raising a validation error is normally the library protecting you correctly; the instinctive "fix" of disabling the check is almost always the real vulnerability, a pattern that echoes Unit 14's lesson about disabled certificate validation.

9.2 Glossary

Term	Definition
Access token	The credential a client presents to a resource server on each API call; typically short-lived
alg (algorithm)	The JWT header field naming the signing algorithm; must never be trusted blindly by a verifier
Audience (aud)	The claim identifying which resource server/API a token is intended for
Authorization Code flow	The OAuth 2.0 flow where the client receives a short-lived code via redirect, then exchanges it server-side for a token
Authorization Server	The OAuth role that authenticates the user and issues tokens
Bearer token	A token that grants access to whoever presents it, like a ticket rather than an ID card
Claim	A single piece of data inside a JWT payload (e.g. sub, exp, scope)
Client (OAuth)	The application requesting access on the resource owner's behalf
JWKS	JSON Web Key Set; a published document of public keys, referenced by kid, used to verify asymmetrically signed tokens

Term	Definition
JWT (JSON Web Token)	A compact, signed, base64url-encoded set of claims (RFC 7519)
kid (key ID)	A header field identifying which key (from a JWKS) was used to sign a token
OpenID Connect (OIDC)	An identity layer built on top of OAuth 2.0 that adds user authentication
PKCE	Proof Key for Code Exchange; proves the party redeeming an authorization code is the same party that requested it
Redirect URI	The exact, pre-registered URL the authorization server returns the user to with the authorization code
Refresh token	A longer-lived credential used to obtain new access tokens without re-authenticating
Resource Owner	The user who owns the data and grants access to it
Resource Server	The API/service that hosts protected data and validates access tokens
Scope	A named, limited permission requested by the client and consented to by the user
State	A random value used to protect the OAuth flow itself against CSRF

9.3 Self-study and further reading

- **RFC 6749** (The OAuth 2.0 Authorization Framework) and **RFC 7636** (PKCE) - the primary specifications.
- **RFC 7519** (JSON Web Token) and **RFC 8725** (JSON Web Token Best Current Practices) - the latter is written almost entirely around pitfalls like the ones in Section 6.
- **OWASP Cheat Sheet Series**: "JSON Web Token for Java", "OAuth2".
- **OWASP API Security Top 10** (Unit 6): API2 Broken Authentication and API5 Broken Function Level Authorization map directly onto this unit.
- **jwt.io** for interactive, safe decoding of your own tokens during development (remember: decoding is not the same as verifying).
- **OAuth 2.1** (IETF Internet-Draft; check current status) for where the specification is heading.
- **Practice platforms (lab only):** OWASP Juice Shop and the PortSwigger Web Security Academy both include dedicated JWT and OAuth challenges.

Appendix A: Full Lab Code Listings

These files were written and verified while preparing this unit (all scenarios produced the output quoted throughout). Copy each listing into a file of the same name in one folder, alongside the RSA key pair from A.7.

A.1 auth_server.py

```
# auth_server.py -- a minimal OAuth 2.0 Authorization Server. LAB USE ONLY, not production
code.
# Implements: Authorization Code flow with PKCE (S256), RS256-signed JWT access tokens,
# a JWKS endpoint so resource servers can fetch the public key by "kid".
# Run:  python auth_server.py      ->  http://127.0.0.1:7000
import base64
import hashlib
import json
import secrets
import time

import jwt
from flask import Flask, request, redirect, jsonify, render_template_string

app = Flask(__name__)

PRIVATE_KEY = open("as_private.pem").read()
PUBLIC_KEY = open("as_public.pem").read()
KID = "as-key-1"
ISSUER = "http://127.0.0.1:7000"

# --- "database" (in-memory, for the lab only) ---
REGISTERED_CLIENTS = {
    "minibank-app": {"redirect_uris": ["http://127.0.0.1:7001/callback"]}
}
USERS = {"alice": "alice123"}
AUTH_CODES = {}     # code -> {client_id, redirect_uri, code_challenge, user, aud, scope,
expires}

LOGIN_PAGE = """<h2>MiniBank Identity Provider</h2>
<p>The app <b>{{ client_id }}</b> wants to access: <b>{{ scope }}</b></p>
<form method="POST">
  <input type="hidden" name="client_id" value="{{ client_id }}">
  <input type="hidden" name="redirect_uri" value="{{ redirect_uri }}">
  <input type="hidden" name="state" value="{{ state }}">
  <input type="hidden" name="code_challenge" value="{{ code_challenge }}">
  <input type="hidden" name="code_challenge_method" value="{{ code_challenge_method }}">
  <input type="hidden" name="scope" value="{{ scope }}">
  <input name="username" placeholder="username" value="alice">
  <input name="password" type="password" placeholder="password" value="alice123">
  <button>Log in and authorise</button>
</form>"""


@app.route("/authorize", methods=["GET", "POST"])
def authorize():
    if request.method == "GET":
        client_id = request.args.get("client_id")
        redirect_uri = request.args.get("redirect_uri")
        state = request.args.get("state", "")
        code_challenge = request.args.get("code_challenge", "")
        code_challenge_method = request.args.get("code_challenge_method", "")
```

```
        scope = request.args.get("scope", "accounts:read")

        client = REGISTERED_CLIENTS.get(client_id)
        if not client or redirect_uri not in client["redirect_uris"]:
            return "Unknown client_id or redirect_uri not registered", 400
        if code_challenge_method != "S256" or not code_challenge:
            return "PKCE required: code_challenge (S256) missing", 400

        return render_template_string(LOGIN_PAGE, client_id=client_id,
redirect_uri=redirect_uri,
                                      state=state, code_challenge=code_challenge,
                                      code_challenge_method=code_challenge_method, scope=scope)

    # POST: user logged in and approved
    u = request.form.get("username", "")
    pw = request.form.get("password", "")
    if USERS.get(u) != pw:
        return "Invalid login", 401

    code = secrets.token_urlsafe(24)
    AUTH_CODES[code] = {
        "client_id": request.form["client_id"],
        "redirect_uri": request.form["redirect_uri"],
        "code_challenge": request.form["code_challenge"],
        "user": u,
        "scope": request.form.get("scope", "accounts:read"),
        "expires": time.time() + 60,         # codes are single-use and short-lived
        "used": False,
    }
    qs = f"code={code}&state={request.form.get('state', '')}"
    return redirect(f"{request.form['redirect_uri']}?{qs}")


@app.route("/token", methods=["POST"])
def token():
    grant_type = request.form.get("grant_type")
    if grant_type != "authorization_code":
        return jsonify(error="unsupported_grant_type"), 400

    code = request.form.get("code")
    verifier = request.form.get("code_verifier", "")
    redirect_uri = request.form.get("redirect_uri")
    client_id = request.form.get("client_id")

    entry = AUTH_CODES.get(code)
    if not entry or entry["used"] or entry["expires"] < time.time():
        return jsonify(error="invalid_grant", error_description="code unknown, used, or
expired"), 400
    if entry["client_id"] != client_id or entry["redirect_uri"] != redirect_uri:
        return jsonify(error="invalid_grant", error_description="client_id/redirect_uri
mismatch"), 400

    # --- PKCE verification: THIS is what stops a stolen authorization code from being redeemed
---
    computed =
base64.urlsafe_b64encode(hashlib.sha256(verifier.encode()).digest()).rstrip(b"=").decode()
    if computed != entry["code_challenge"]:
        return jsonify(error="invalid_grant", error_description="PKCE verification failed"),
400

    entry["used"] = True   # one-time use
```

```
    now = int(time.time())
    access_payload = {
        "iss": ISSUER,
        "sub": entry["user"],
        "aud": "http://127.0.0.1:7002",      # the ONE resource server this token is for
        "iat": now,
        "exp": now + 300,                     # short-lived access token: 5 minutes
        "scope": entry["scope"],
        "client_id": client_id,
    }
    access_token = jwt.encode(access_payload, PRIVATE_KEY, algorithm="RS256", headers={"kid":
KID})
    return jsonify(access_token=access_token, token_type="Bearer", expires_in=300,
                   scope=entry["scope"])


@app.route("/.well-known/jwks.json")
def jwks():
    # Expose the PUBLIC key only, in JWKS format, so resource servers can verify signatures
    # without ever holding the authorization server's private key.
    from cryptography.hazmat.primitives import serialization
    pub = serialization.load_pem_public_key(PUBLIC_KEY.encode())
    numbers = pub.public_numbers()

    def b64(n, length):
        return base64.urlsafe_b64encode(n.to_bytes(length, "big")).rstrip(b"=").decode()

    jwk = {
        "kty": "RSA", "kid": KID, "use": "sig", "alg": "RS256",
        "n": b64(numbers.n, 256), "e": b64(numbers.e, 3),
    }
    return jsonify(keys=[jwk])


if __name__ == "__main__":
    app.run(host="127.0.0.1", port=7000)
```

A.2 client_app.py

```
# client_app.py -- the OAuth 2.0 "Relying Party" / client application. LAB USE ONLY.
# Starts the Authorization Code + PKCE flow and exchanges the code for a token.
# Run:  python client_app.py      ->  http://127.0.0.1:7001
import base64
import hashlib
import secrets
import urllib.parse

import requests
from flask import Flask, request, redirect, session, render_template_string

app = Flask(__name__)
app.secret_key = "lab-only-secret"

AUTH_SERVER = "http://127.0.0.1:7000"
CLIENT_ID = "minibank-app"
REDIRECT_URI = "http://127.0.0.1:7001/callback"
```

```
@app.route("/")
def home():
    token = session.get("access_token")
    if not token:
        return '<a href="/login">Log in with MiniBank</a>'
    return render_template_string(
        "<h2>Logged in</h2><p>Access token (first 40 chars):<br>{{ t }}...</p>"
        '<p><a href="/call-api">Call the protected API</a></p>',
        t=token[:40])



@app.route("/login")
def login():
    # --- PKCE: generate a random verifier, derive the challenge, keep the verifier secret ---
    verifier = base64.urlsafe_b64encode(secrets.token_bytes(32)).rstrip(b"=").decode()
    challenge =
base64.urlsafe_b64encode(hashlib.sha256(verifier.encode()).digest()).rstrip(b"=").decode()
    state = secrets.token_urlsafe(16)
    session["pkce_verifier"] = verifier
    session["oauth_state"] = state

    params = {
        "client_id": CLIENT_ID, "redirect_uri": REDIRECT_URI, "state": state,
        "code_challenge": challenge, "code_challenge_method": "S256",
        "scope": "accounts:read", "response_type": "code",
    }
    return redirect(f"{AUTH_SERVER}/authorize?{urllib.parse.urlencode(params)}")



@app.route("/callback")
def callback():
    if request.args.get("state") != session.get("oauth_state"):
        return "State mismatch -- possible CSRF on the OAuth flow. Aborting.", 400
    code = request.args.get("code")
    resp = requests.post(f"{AUTH_SERVER}/token", data={
        "grant_type": "authorization_code", "code": code,
        "redirect_uri": REDIRECT_URI, "client_id": CLIENT_ID,
        "code_verifier": session.pop("pkce_verifier"),
    })
    if resp.status_code != 200:
        return f"Token exchange failed: {resp.text}", 400
    session["access_token"] = resp.json()["access_token"]
    return redirect("/")



@app.route("/call-api")
def call_api():
    token = session.get("access_token")
    resp = requests.get("http://127.0.0.1:7002/accounts",
                        headers={"Authorization": f"Bearer {token}"})
    return f"<pre>Resource server said ({resp.status_code}):\n{resp.text}</pre>"



if __name__ == "__main__":
    app.run(host="127.0.0.1", port=7001)
```

A.3 resource_server.py

```
# resource_server.py -- a protected API that validates JWT access tokens PROPERLY.
# LAB USE ONLY. Fetches the authorization server's public key via JWKS and pins
```

```
# the algorithm, issuer and audience -- this is the combination that defeats the
# pitfalls demonstrated in jwt_pitfalls.py.
# Run:  python resource_server.py      ->  http://127.0.0.1:7002
import jwt
import requests
from flask import Flask, request, jsonify

app = Flask(__name__)

AUTH_SERVER = "http://127.0.0.1:7000"
EXPECTED_ISSUER = "http://127.0.0.1:7000"
EXPECTED_AUDIENCE = "http://127.0.0.1:7002"

ACCOUNTS = {"alice": [{"id": "AC-001", "balance": 1000}]}

_jwks_cache = None


def get_signing_key(kid):
    global _jwks_cache
    if _jwks_cache is None:
        _jwks_cache = requests.get(f"{AUTH_SERVER}/.well-known/jwks.json").json()
    for k in _jwks_cache["keys"]:
        if k["kid"] == kid:
            return jwt.PyJWK(k).key
    raise jwt.InvalidKeyError(f"No key found for kid={kid}")


@app.route("/accounts")
def accounts():
    auth = request.headers.get("Authorization", "")
    if not auth.startswith("Bearer "):
        return jsonify(error="missing bearer token"), 401
    token = auth[len("Bearer "):]

    try:
        unverified_header = jwt.get_unverified_header(token)
        key = get_signing_key(unverified_header.get("kid"))
        claims = jwt.decode(
            token, key,
            algorithms=["RS256"],          # <-- PIN the algorithm, never trust the token's own
header
            audience=EXPECTED_AUDIENCE,    # <-- this token must be FOR this API
            issuer=EXPECTED_ISSUER,        # <-- and must come FROM our trusted authorization
server
        )
    except jwt.ExpiredSignatureError:
        return jsonify(error="token expired"), 401
    except jwt.InvalidTokenError as e:
        return jsonify(error="invalid token", detail=str(e)), 401

    user = claims["sub"]
    return jsonify(user=user, scope=claims["scope"], accounts=ACCOUNTS.get(user, []))


if __name__ == "__main__":
    app.run(host="127.0.0.1", port=7002)
```

A.4 run_oauth_demo.py

```
# run_oauth_demo.py -- runs the full Authorization Code + PKCE flow against the
# live auth_server.py and resource_server.py, acting as the browser/client would.
# Start both servers first (see run_lab_demo.sh), then run this script.
import base64, hashlib, secrets, re, sys
import requests

AUTH = "http://127.0.0.1:7000"
RS = "http://127.0.0.1:7002"
CLIENT_ID = "minibank-app"
REDIRECT = "http://127.0.0.1:7001/callback"



def step(n, title):
    print(f"\n[{n}] {title}")



verifier = base64.urlsafe_b64encode(secrets.token_bytes(32)).rstrip(b"=").decode()
challenge =
base64.urlsafe_b64encode(hashlib.sha256(verifier.encode()).digest()).rstrip(b"=").decode()

step(1, "Client generates PKCE verifier/challenge and sends the user to /authorize")
r = requests.get(f"{AUTH}/authorize", params={
    "client_id": CLIENT_ID, "redirect_uri": REDIRECT, "state": "xyz",
    "code_challenge": challenge, "code_challenge_method": "S256", "scope": "accounts:read"})
print(f"    GET /authorize -> {r.status_code} (login/consent page shown)")

step(2, "User logs in and approves -- authorization server issues a one-time code")
r2 = requests.post(f"{AUTH}/authorize", data={
    "client_id": CLIENT_ID, "redirect_uri": REDIRECT, "state": "xyz",
    "code_challenge": challenge, "code_challenge_method": "S256", "scope": "accounts:read",
    "username": "alice", "password": "alice123"}, allow_redirects=False)
code = re.search(r"code=([^&]+)", r2.headers["Location"]).group(1)
print(f"    POST /authorize -> {r2.status_code}, redirected with code={code[:12]}...")

step(3, "Client exchanges the code for an access token, proving it holds the PKCE verifier")
r3 = requests.post(f"{AUTH}/token", data={
    "grant_type": "authorization_code", "code": code,
    "redirect_uri": REDIRECT, "client_id": CLIENT_ID, "code_verifier": verifier})
token = r3.json()["access_token"]
print(f"    POST /token -> {r3.status_code}")
print(f"    access_token = {token[:50]}...")

step(4, "Client calls the protected API with the access token")
r4 = requests.get(f"{RS}/accounts", headers={"Authorization": f"Bearer {token}"})
print(f"    GET /accounts -> {r4.status_code}  {r4.json()}")

step(5, "Attacker intercepts a FRESH authorization CODE and tries to redeem it "
        "without the (secret, browser-held) verifier")
r5a = requests.post(f"{AUTH}/authorize", data={
    "client_id": CLIENT_ID, "redirect_uri": REDIRECT, "state": "xyz",
    "code_challenge": challenge, "code_challenge_method": "S256", "scope": "accounts:read",
    "username": "alice", "password": "alice123"}, allow_redirects=False)
stolen_code = re.search(r"code=([^&]+)", r5a.headers["Location"]).group(1)
r5 = requests.post(f"{AUTH}/token", data={
    "grant_type": "authorization_code", "code": stolen_code,
    "redirect_uri": REDIRECT, "client_id": CLIENT_ID, "code_verifier": "guessed-wrong"})
print(f"    POST /token (correct fresh code, WRONG verifier) -> {r5.status_code}  {r5.json()}")

step(6, "Legitimate client tries to reuse its OWN already-spent code a second time")
```

```
r6 = requests.post(f"{AUTH}/token", data={
    "grant_type": "authorization_code", "code": code,
    "redirect_uri": REDIRECT, "client_id": CLIENT_ID, "code_verifier": verifier})
print(f"    POST /token (code reuse) -> {r6.status_code}  {r6.json()}")

print("\nDemo complete: PKCE blocked a stolen/guessed code, and the code was single-use.")
```

A.5 jwt_inspect.py

```
# jwt_inspect.py -- decode a JWT by hand, no library, to show there is NO secret
# needed to READ a JWT's header and payload. Run: python jwt_inspect.py <token>
import base64
import json
import sys



def b64url_decode(segment: str) -> bytes:
    padding = "=" * (-len(segment) % 4)
    return base64.urlsafe_b64decode(segment + padding)



def main():
    token = sys.argv[1] if len(sys.argv) > 1 else open("sample.jwt").read().strip()
    parts = token.split(".")
    if len(parts) != 3:
        print("Not a JWT (expected 3 dot-separated parts)")
        return
    header_b64, payload_b64, signature_b64 = parts

    header = json.loads(b64url_decode(header_b64))
    payload = json.loads(b64url_decode(payload_b64))

    print("=== HEADER (base64url-decoded, anyone can read this) ===")
    print(json.dumps(header, indent=2))
    print("\n=== PAYLOAD (base64url-decoded, anyone can read this) ===")
    print(json.dumps(payload, indent=2))
    print(f"\n=== SIGNATURE ===\n{len(b64url_decode(signature_b64))} raw bytes "
          f"-- this is the ONLY part that proves the token is genuine")
    print("\nNOTE: base64url is an ENCODING, not ENCRYPTION. No key was needed")
    print("to read everything above. A JWT's contents are PUBLIC; only its")
    print("SIGNATURE is secret. Never put secrets you don't want read inside a JWT.")



if __name__ == "__main__":
    main()
```

A.6 jwt_pitfalls.py

```
# jwt_pitfalls.py -- demonstrates common JWT verification MISTAKES, each as a real,
# runnable exploit against a VULNERABLE verifier, followed by the SAME exploit
# failing against a SECURE verifier. LAB USE ONLY.
#
# Run:  python jwt_pitfalls.py
import base64
import hashlib
import hmac as hmac_mod
import json
import time
```

```
import jwt

PRIVATE_KEY = open("as_private.pem").read()
PUBLIC_KEY = open("as_public.pem").read()



def b64url(data: bytes) -> str:
    return base64.urlsafe_b64encode(data).rstrip(b"=").decode()



def genuine_token(include_aud=True, **overrides):
    now = int(time.time())
    payload = {"sub": "alice", "role": "user", "iss": "https://auth.minibank.local",
               "iat": now, "exp": now + 300}
    if include_aud:
        payload["aud"] = "https://api.minibank.local"
    payload.update(overrides)
    return jwt.encode(payload, PRIVATE_KEY, algorithm="RS256", headers={"kid": "as-key-1"})



# ======================================================================
# PITFALL 1: alg=none -- accepting a token that claims to need no signature
# ======================================================================
def pitfall_1():
    print("\n--- PITFALL 1: alg=none ---")

    def vulnerable_verify(token):
        # BUG: decodes using whatever algorithm the TOKEN ITSELF claims in its header
        header = jwt.get_unverified_header(token)
        return jwt.decode(token, options={"verify_signature": False}) if header["alg"] ==
"none" \
            else jwt.decode(token, PUBLIC_KEY, algorithms=[header["alg"]])

    def secure_verify(token):
        # FIX: the ALLOWED algorithm list is fixed by the SERVER, never read from the token
        return jwt.decode(token, PUBLIC_KEY, algorithms=["RS256"])

    header = b64url(json.dumps({"alg": "none", "typ": "JWT"}).encode())
    payload = b64url(json.dumps({"sub": "attacker", "role": "admin",
                                 "exp": int(time.time()) + 3600}).encode())
    forged_token = f"{header}.{payload}."       # note: empty signature segment

    try:
        claims = vulnerable_verify(forged_token)
        print(f"  VULNERABLE verifier: ACCEPTED forged token -> role={claims['role']}  ***
EXPLOIT WORKS ***")
    except Exception as e:
        print(f"  VULNERABLE verifier: rejected ({e})  -- unexpected, check the code")

    try:
        secure_verify(forged_token)
        print("  SECURE verifier: ACCEPTED forged token -- THIS SHOULD NOT HAPPEN")
    except jwt.InvalidTokenError as e:
        print(f"  SECURE verifier: REJECTED forged token ({type(e).__name__}: {e})  -- attack
blocked")



# ======================================================================
# PITFALL 2: RS256/HS256 "algorithm confusion" (key confusion)
```

```
# ======================================================================
def pitfall_2():
    print("\n--- PITFALL 2: RS256/HS256 algorithm confusion ---")
    print("  (modern PyJWT (2.x) actually blocks this by default if you pass a PEM string to")
    print("   algorithms=[\"RS256\",\"HS256\"] -- a real defence-in-depth improvement. The
bug")
    print("   below is written the way many hand-rolled or older verifiers still do it: by")
    print("   HAND-checking the signature with hmac, trusting the 'alg' the token itself
claims.)")

    def vulnerable_verify_by_hand(token, public_key_pem):
        # BUG: trusts the token's OWN header to decide how to verify it, and for "HS256"
        # treats the (public, publishable) RSA key bytes as if they were a private HMAC secret
        h_b64, p_b64, s_b64 = token.split(".")
        header = json.loads(base64.urlsafe_b64decode(h_b64 + "=="))
        signing_input = f"{h_b64}.{p_b64}".encode()
        sig = base64.urlsafe_b64decode(s_b64 + "==")
        if header["alg"] == "HS256":
            expected = hmac_mod.new(public_key_pem.encode(), signing_input,
hashlib.sha256).digest()
            if not hmac_mod.compare_digest(sig, expected):
                raise jwt.InvalidSignatureError("bad HMAC signature")
        else:
            jwt.decode(token, public_key_pem, algorithms=["RS256"])  # delegate RS256 properly
        return json.loads(base64.urlsafe_b64decode(p_b64 + "=="))

    def secure_verify(token, public_key_pem):
        # FIX: only RS256 is ever allowed for tokens meant to be asymmetrically signed,
        # and verification is delegated entirely to a trusted library -- never hand-rolled
        return jwt.decode(token, public_key_pem, algorithms=["RS256"])

    public_key_bytes = PUBLIC_KEY.encode()
    header = {"alg": "HS256", "typ": "JWT"}
    payload = {"sub": "attacker", "role": "admin", "exp": int(time.time()) + 3600}
    h_b64 = b64url(json.dumps(header).encode())
    p_b64 = b64url(json.dumps(payload).encode())
    signing_input = f"{h_b64}.{p_b64}".encode()
    forged_sig = hmac_mod.new(public_key_bytes, signing_input, hashlib.sha256).digest()
    forged_token = f"{h_b64}.{p_b64}.{b64url(forged_sig)}"

    try:
        claims = vulnerable_verify_by_hand(forged_token, PUBLIC_KEY)
        print(f"  VULNERABLE hand-rolled verifier: ACCEPTED forged token ->
role={claims['role']}  *** EXPLOIT WORKS ***")
    except Exception as e:
        print(f"  VULNERABLE hand-rolled verifier: rejected ({e})")

    try:
        secure_verify(forged_token, PUBLIC_KEY)
        print("  SECURE verifier: ACCEPTED forged token -- THIS SHOULD NOT HAPPEN")
    except jwt.InvalidTokenError as e:
        print(f"  SECURE verifier: REJECTED forged token ({type(e).__name__}: {e})  -- attack
blocked")



# ======================================================================
# PITFALL 3: not checking expiry (or not checking it at all)
# ======================================================================
def pitfall_3():
    print("\n--- PITFALL 3: ignoring token expiry ---")
```

```
    expired_token = genuine_token(include_aud=False, exp=int(time.time()) - 3600)   # expired
one hour ago

    def vulnerable_verify(token):
        # BUG: verify_exp explicitly turned off (seen in real code "to make testing easier")
        return jwt.decode(token, PUBLIC_KEY, algorithms=["RS256"],
                          options={"verify_exp": False})

    def secure_verify(token):
        return jwt.decode(token, PUBLIC_KEY, algorithms=["RS256"])   # exp checked by default

    claims = vulnerable_verify(expired_token)
    print(f"  VULNERABLE verifier: ACCEPTED an hour-old EXPIRED token -> sub={claims['sub']}
*** EXPLOIT WORKS ***")
    try:
        secure_verify(expired_token)
        print("  SECURE verifier: ACCEPTED expired token -- THIS SHOULD NOT HAPPEN")
    except jwt.ExpiredSignatureError as e:
        print(f"  SECURE verifier: REJECTED expired token ({e})  -- attack blocked")



# ======================================================================
# PITFALL 4: weak / guessable HMAC secret
# ======================================================================
def pitfall_4():
    print("\n--- PITFALL 4: weak HMAC secret (brute-forceable) ---")

    WEAK_SECRET = "secret123"    # a real secret someone actually used
    token = jwt.encode({"sub": "alice", "role": "user", "exp": int(time.time()) + 3600},
                       WEAK_SECRET, algorithm="HS256")
    print(f"  A token signed with a short, common HS256 secret: {token[:50]}...")

    candidates = ["password", "123456", "changeme", "minibank", "letmein", "secret123",
"admin"]
    cracked = None
    for guess in candidates:
        try:
            jwt.decode(token, guess, algorithms=["HS256"])
            cracked = guess
            break
        except jwt.InvalidSignatureError:
            continue
    if cracked:
        forged = jwt.encode({"sub": "attacker", "role": "admin", "exp": int(time.time()) +
3600},
                            cracked, algorithm="HS256")
        claims = jwt.decode(forged, cracked, algorithms=["HS256"])
        print(f"  Secret CRACKED from a {len(candidates)}-word list: {cracked!r}")
        print(f"  Attacker now MINTS their own valid token -> role={claims['role']}  ***
EXPLOIT WORKS ***")
    print("  FIX: use a long, random secret (32+ bytes from a CSPRNG), or prefer RS256/ES256")
    print("       (asymmetric signing) so the verifying party never needs the signing secret at
all.")



# ======================================================================
# PITFALL 5: missing audience / issuer check (token confusion between services)
# ======================================================================
def pitfall_5():
    print("\n--- PITFALL 5: missing audience/issuer check ---")

    token_for_other_service = genuine_token(aud="https://notifications.minibank.local")
```

```
    def vulnerable_verify(token):
        # BUG: a developer hit PyJWT's "Invalid audience" error during testing (because no
        # audience= was supplied) and "fixed" it by turning the check off -- a very real,
        # very common pattern -- instead of supplying the correct expected audience.
        return jwt.decode(token, PUBLIC_KEY, algorithms=["RS256"],
                          options={"verify_aud": False})

    def secure_verify(token):
        # FIX: pin the expected audience AND issuer -- never disable the check
        return jwt.decode(token, PUBLIC_KEY, algorithms=["RS256"],
                          audience="https://api.minibank.local",
                          issuer="https://auth.minibank.local")

    claims = vulnerable_verify(token_for_other_service)
    print(f"  VULNERABLE verifier (accounts API): ACCEPTED a token meant for "
          f"{claims['aud']!r}  *** EXPLOIT WORKS (token confusion) ***")
    try:
        secure_verify(token_for_other_service)
        print("  SECURE verifier: ACCEPTED mismatched-audience token -- THIS SHOULD NOT
HAPPEN")
    except jwt.InvalidAudienceError as e:
        print(f"  SECURE verifier: REJECTED mismatched-audience token ({e})  -- attack
blocked")


if __name__ == "__main__":
    pitfall_1()
    pitfall_2()
    pitfall_3()
    pitfall_4()
    pitfall_5()
    print("\nAll five pitfalls demonstrated: each VULNERABLE verifier was fooled,")
    print("and each SECURE verifier correctly rejected the same forged/misused token.")
```

A.7 Generating the lab RSA key pair

Verified to run cleanly with OpenSSL 3.x

```
# Run once, from the lab folder, before using any of the scripts above.
openssl genrsa -out as_private.pem 2048
openssl rsa -in as_private.pem -pubout -out as_public.pem
```

A.8 make_sample_jwt.py

```
# make_sample_jwt.py -- creates sample.jwt, a realistic RS256 token, for jwt_inspect.py
import jwt, datetime
priv = open("as_private.pem").read()
now = datetime.datetime.utcnow()
payload = {
    "iss": "https://auth.minibank.local", "sub": "user-1001",
    "aud": "https://api.minibank.local",
    "iat": int(now.timestamp()), "exp": int((now + datetime.timedelta(minutes=5)).timestamp()),
    "scope": "accounts:read", "name": "Alice",
}
token = jwt.encode(payload, priv, algorithm="RS256", headers={"kid": "as-key-1"})
open("sample.jwt", "w").write(token)
```

A.9 run_lab_demo.sh

```
#!/bin/bash
# run_lab_demo.sh -- runs the OAuth 2.0 + PKCE flow, then the JWT decode and
# pitfalls demonstrations, in sequence. LAB USE ONLY. All traffic stays on 127.0.0.1.
set -e
cd "$(dirname "$0")"

echo "Starting authorization server (7000) and resource server (7002)..."
python3 auth_server.py >/tmp/auth_server.log 2>&1 &
PID1=$!
python3 resource_server.py >/tmp/resource_server.log 2>&1 &
PID2=$!
sleep 1.5

echo
echo "### PART 1: OAuth 2.0 Authorization Code flow with PKCE ###"
python3 run_oauth_demo.py

kill $PID1 $PID2 2>/dev/null
sleep 1

echo
echo "### PART 2: Decoding a JWT by hand (no library, no secret needed) ###"
python3 jwt_inspect.py

echo
echo "### PART 3: Five common JWT verification pitfalls ###"
python3 jwt_pitfalls.py

echo
echo "Demo complete."
```

Verified result when run: output matches Sections 4.1 and 6.1–6.5 exactly (captured during preparation of this unit).